

Requirements Analysis for Running Dashboard Tool



Personal Background and Experience

- **Work Experience**

- Data Scientist, Software Developer, and AWS Certified Solutions Architect with 4 years of experience developing analytical and software solutions in industry and academia.

- **Background**

- Computational Engineering alumni of University of Texas at Austin

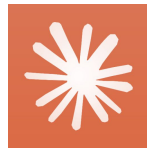
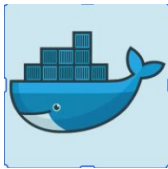
- **Highlights**

- Cost Estimating Relationship Tool that streamlines the end-to-end model development workflow improving model turnaround time, maintainability, and auditability
 - Inferential Statistical Analysis Study driving insights into cost overruns in defense contracts for program risk management
- 

Personal Background and Experience

Technical Toolkit

- Programming: Python, SQL, R, C++, Bash
- Data & Analytics: Python (Pandas, NumPy, SciPy, Scikit-Learn), R, Jupyter Notebook
- Cloud & DevOps: AWS, Docker, Kubernetes, Git/GitHub, Cursor, Claude, GitHub Copilot
- Databases & Project Management: Redis NoSQL, MySQL, MSSQL Server, Agile, Jira



Why Perform Requirements Analysis?

- **Background:** Project is being developed as a solo-effort with no mandatory requirements, milestones, or deliverables
- **Initial Status:** Initial documentation of requirements in the task plan represents a collection of initial ideas for the design and function of the system
- **Purpose:**
 - Constrain the scope of the project into a smaller list of requirements that are more achievable and timely
 - Categorize list of requirements to enable prioritization
 - Investigate trade-offs to be informed of future design decisions and their impacts

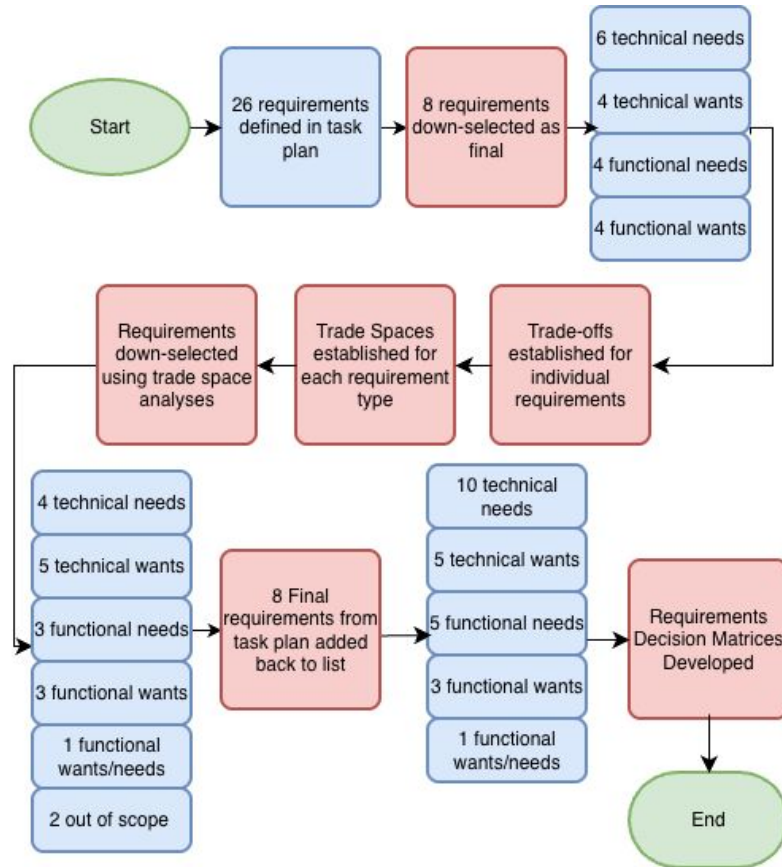


How was this accomplished?

- 1) Initial review performed to separate finalized requirements from requirements for further analysis
- 2) Trade-offs established for individual requirements
- 3) Trade Spaces developed for each requirement type (functional vs technical, needs vs wants)
- 4) Requirements down-selected using trade space analyses
- 5) Requirements Decision Matrices developed to visualize association between customer satisfaction and technical difficulty



Requirements Analysis Process



Analysis for Example Requirement: Step #1: Trade-offs for Individual Requirements

Functional Requirements - Nice to Have

1) Interactive Visualization Features in GUI

- **Long Description:** Enabling visualizations that support interactive zoom, pan, and interaction features improves user experience increasing use cases and customization of dashboard allowing for drilled-down investigation of trends, however, this may require increased development time, and possibly moderate reduction in application UI responsiveness which may be too significant of a trade-off especially considering that the user can use browser features or desktop image-viewing features to zoom in and out of visualizations.
- **Core Trade-Offs:**
 - User Experience Quality, Use Case Coverage, and Dashboard Customization vs Development Time and UI Responsiveness
- **Trade-Off Summary:**
 - **Strengths**
 - Improved User Experience Quality through interactive exploration
 - Increased Use Case Coverage enabling deeper trend investigation
 - Increased Dashboard Customization through flexible viewing interactions
 - Improved analytical capability through drill-down visualization
 - Reduced need for external viewing tools
 - **Limitations**
 - Increased Development Time required to implement interaction logic
 - Reduced UI Responsiveness due to additional rendering demands
 - Increased UI complexity affecting responsiveness stability
 - Potential redundancy with browser-native zoom features

• Trade-Off Compromises:

- **Increasing User Experience Quality**
 - ↑ Development Time
 - ↓ UI Responsiveness
- **Increasing Use Case Coverage**
 - ↑ Development Time
 - ↓ UI Responsiveness
- **Increasing Dashboard Customization**
 - ↑ Development Time
 - ↓ UI Responsiveness
- **Increasing UI Responsiveness**
 - ↓ Dashboard Customization
 - ↓ User Experience Quality
- **Increasing Development Time**
 - Enables ↑ Dashboard Customization
 - Enables ↑ Use Case Coverage

Step #2: Trade Spaces Established for each Requirement Type

1) Trade Space 1: Feature Capability

a) Trade-Offs Pulling on Each Other:

- i) User Capability
- ii) Development Time
- iii) UI Responsiveness

b) Trade-Off Relationships:

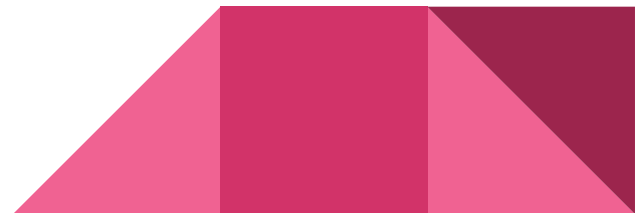
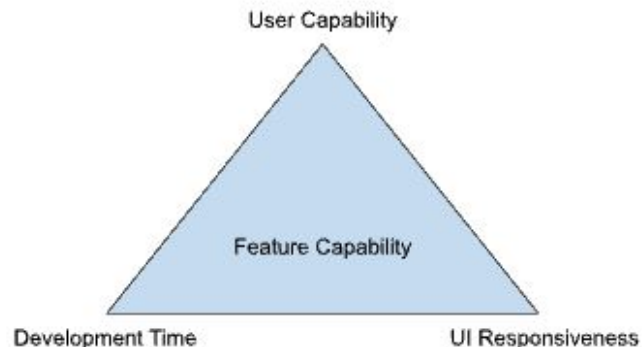
- i) Increasing User Capability
↑ Development Time
↓ UI Responsiveness
- ii) Increasing UI Responsiveness
↓ User Capability
- iii) Increasing Development Time
Enables ↑ User Capability

c) Importance of Schematic:

- i) This diagram covers feature-driven UX growth including user experience quality, use case coverage, and dashboard customization competing with engineering and performance limits. This trade space represents UX sophistication, complexity scaling, and feature latency.

d) Source Requirements Covered:

- i) Interactive Visualization Features
- ii) Global Time-Range Selector
- iii) Drag-and-Drop Upload
- iv) Configurable Settings Panel



Step #3: Requirements Down-Selected using Trade Space Analysis

Trade-Off Analysis & Downselected Requirements

- a) **Req F2a Analysis** - The requirement that plots on the GUI be interactive has implications on the Feature Capability trade space. Enabling visualizations that support interactive zoom, pan, and other interaction features improves user experience, increasing use cases and customization of the dashboard allowing for drilled-down investigation of trends. However, this may require increased development time, and possibly moderate reduction in application UI responsiveness which may be too significant of a trade-off especially considering that the user can use browser features or desktop image-viewing features to zoom in and out of visualizations.
- i) **Decision** - Since enabling interactivity in the visualizations adds very little benefit to the utility of the dashboard, would decrease UI responsiveness, require additional development time, and can be substituted for native image viewing tools, this requirement is being down-selected as a **want**.



Technical Requirements Decision Matrix

Customer Satisfaction\Technology Differentiation	Easy to Do	Hard to Do
Needs	(Req TN1 - TN3) Tech Stack: Python, Cursor, Git/GitHub (Req TN4 + TN5): Error handling for filetype and fields (Req TN6): Custom file upload into repository path	(Req TN7): Comprehensive system testing (Req TN8): Derive computed fields from .gpx (Req TN9): Personalized time-series forecasting models (generic) (Req TN10): Tailored race time predictions
Wants	(Req TW1): Error handling for invalid UI inputs (Req TW2): Data persistence for future launches	Req TW3): Tailored training load insights (Req TW4): Anomalous run reports (Req TW5): Personalized target race date

Functional Requirements Decision Matrix

Customer Satisfaction\Technology Differentiation	Easy to Do	Hard to Do
Needs	(Req FN2): Agile Kanban for PM (Req FN3): GitHub versioning and releases for MacOSXX (Req FN4): Dashboard as standalone application (Req FN6): Calculate and display common metrics	(Req FN1): Single-Page dashboard display (Req FN5): Optional GUI data export (Req FN6): Calculate and display common indicators + KPIs
Wants	(Req FW1): Plot interactivity (Req FW3): Drag-and-drop file upload	(Req FW2): Global date-range selector for UI (Req FW4): Calculate and display uncommon indicators + KPIs

Next Steps

- **Preliminary System Design (Platform-Agnostic)**
 - Black Box Diagrams
 - Use Case Workflows
 - System Architectural Diagrams
 - UI Designs
- **Feasibility Analysis Studies**
 - Define categories and sub-categories
 - Perform Literature Review to assemble competing alternatives
 - Define Feasibility Analysis methodology
 - Define and weigh Feasibility Analysis criteria
 - Perform analysis to down-select alternatives

